```
CREATE DATABASE IF NOT EXISTS ass6;

USE ass6;

DROP TABLE IF EXISTS O_RollCall;
DROP TABLE IF EXISTS N_RollCall;

CREATE TABLE O_RollCall (
   id INT PRIMARY KEY,
   name VARCHAR(255) NOT NULL,
   roll_call_date DATE NOT NULL
);
CREATE TABLE N_RollCall (
   id INT PRIMARY KEY,
   name VARCHAR(255) NOT NULL,
   roll_call_date DATE NOT NULL
);

INSERT INTO N_RollCall (id, name, roll_call_date)
VALUES
   (1, 'Amit', '2025-08-01'),
   (2, 'Priya', '2025-08-02'),
   (3, 'Rahul', '2025-08-03'),
   (4, 'Sanya', '2025-08-04'),
   (5, 'Vikram', '2025-08-05'),
   (6, 'Neha', '2025-08-06'),
   (7, 'Arjun', '2025-08-07'),
   (8, 'Kavita', '2025-08-08');

INSERT INTO O_RollCall VALUES (11, 'Sagar', '2005-08-22');

DELIMITER $$

CREATE PROCEDURE MergeRollCallData()
BEGIN
   -- Declare local variables
   DECLARE done INT DEFAULT 0;
   DECLARE n_id INT;
   DECLARE n_name VARCHAR(255);
   DECLARE n_date DATE;

   -- Declare cursor to read all rows from N_RollCall
   DECLARE cur CURSOR FOR
      SELECT id, name, roll_call_date FROM N_RollCall;

   -- Handler to detect end of data
   DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = 1;

   -- Open the cursor
   OPEN cur;

   -- Loop through all rows from new table
   read_loop: LOOP
      FETCH cur INTO n_id, n_name, n_date;

      -- Exit when all rows are processed
      IF done THEN
```

```
        LEAVE read_loop;
      END IF;

      -- Check if record exists in old table before inserting
      IF NOT EXISTS (
        SELECT 1
        FROM O_RollCall
        WHERE id = n_id
          AND name = n_name
          AND roll_call_date = n_date
      ) THEN
        INSERT INTO O_RollCall (id, name, roll_call_date)
        VALUES (n_id, n_name, n_date);
      END IF;
    END LOOP;

    -- Close cursor after completion
    CLOSE cur;
END $$

DELIMITER ;


CALL MergeRollCallData();

SELECT * FROM O_RollCall;
```

```
-- ------------------------------------------------------------------
-- 💬 Viva Preparation Notes
-- ------------------------------------------------------------------
-- 🔹 1️⃣ What is a Stored Procedure?
--    ➤ A stored procedure is a set of SQL statements stored in the database
--      that can be reused and executed as a single unit.
--
-- 🔹 2️⃣ What is a Cursor in MySQL?
--    ➤ A cursor allows row-by-row processing of query results, used inside
--      stored procedures when we need to perform operations on each record.
--
-- 🔹 3️⃣ Why do we use a CONTINUE HANDLER?
--    ➤ It handles runtime conditions or exceptions, such as the "NOT FOUND"
--      condition when a cursor reaches the end of its records.
--
-- 🔹 4️⃣ Why do we use IF NOT EXISTS?
```

```
--   ➤ To prevent duplicate entries while merging data.
--
-- ◆ 5️⃣ What is the role of DELIMITER?
--   ➤ DELIMITER changes the command-ending symbol from ";" to "$$"
--     so MySQL can treat the entire procedure body as one unit.
--
-- ◆ 6️⃣ Difference between PROCEDURE and FUNCTION?
--   ➤ A FUNCTION returns a single value and can be used inside queries,
--     whereas a PROCEDURE performs tasks and is executed using CALL.
--
-- ◆ 7️⃣ Why do we use LOOP in stored procedures?
--   ➤ LOOP allows repeated execution of SQL statements until a condition is met.
--
-- ◆ 8️⃣ What is the purpose of the CURSOR here?
--   ➤ The cursor reads each record from the new roll call table (N_RollCall)
--     one by one and checks if it already exists in the old table.
--
-- ◆ 9️⃣ What happens if we remove the HANDLER?
--   ➤ Without it, the procedure would throw an error when the cursor reaches
--     the last record, stopping execution abruptly.
--
-- ◆ 🔟 How can you modify this code to update existing records?
--   ➤ Add an UPDATE statement inside the loop with a condition on ID.
--
-- ◆ 1️⃣1️⃣ Difference between DECLARE and SET?
--   ➤ DECLARE is used to define a variable, while SET is used to assign a value.
--
-- ◆ 1️⃣2️⃣ What happens if the same ID appears in both tables?
--   ➤ The procedure will skip insertion (due to IF NOT EXISTS) — avoiding duplicates.
--
-- ◆ 1️⃣3️⃣ What is the advantage of using stored procedures?
--   ➤ Reusability, better performance, reduced network traffic, and security.
--
-- ◆ 1️⃣4️⃣ Can we call a stored procedure from another procedure?
--   ➤ Yes, nested procedure calls are allowed in MySQL.
--
-- ◆ 1️⃣5️⃣ What are real-life examples of cursors?
--   ➤ Payroll processing, student result updates, or data migration tasks where
--     record-by-record logic is needed.
--
```